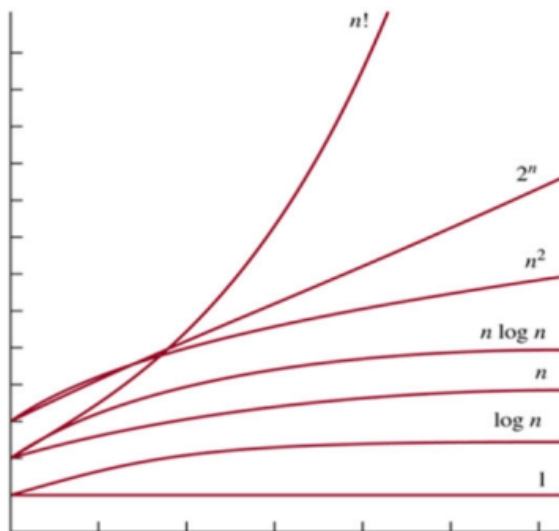


Big Oh Notation

- The stakes are very high when two algorithms try to solve the same problem but scale so differently with the size of the problem (n)
- We want to express this scaling in a way that...
 - Is simple
 - Ignores the differences between different hardware, and other computer processes
 - Ignores special behaviours for small n
- Suppose that the number of “steps” (operations that don’t depend on n) can be expressed as $f(n)$
- We say that $f(n) \in O(g(n))$ if...
 - There are positive constants c and n_0 , such that $f(n) \leq c \cdot g(n)$ for any natural number $n \geq n_0$



Example

For each of the following, find the Big-Oh time complexity

- a) $f(n) = 7n^2$
 - $O(n^2)$
- b) $f(n) = n^2 + 396$
 - $O(n^2)$
- c) $f(n) = 3960n + 4000$
 - $O(n)$

Considerations

- If $f \in O(n)$, then it's also the case that $f \in O(n \log n)$, $f \in O(n^2)$, and all larger bounds
 - $O(1) \subseteq O(\log n) \subseteq O(n) \subseteq O(n^2) \subseteq O(n^3) \subseteq O(2^n) \subseteq O(n!) \subseteq O(n^n)$
 - We commonly refer to O , but often mean θ
 - What we're concerned about is the tightest upper bound
 - So while a function has a worst case running time proportional to $O(n \log n)$ is in $O(n^2)$, we wouldn't say that the running time is $O(n^2)$
-

Big Theta

- We want the tightest bound
 - If an algorithm is $O(n)$, it's also $O(n^2)$
 - A $\theta(n)$ algorithm is not $\theta(n^2)$
 - We say that $f(n) \in \theta(g(n))$ if...
 - There are positive constants c_1, c_2, n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$, for all $n \geq n_0$
-

Example

For each of the following functions, find the dominant term of the function

- a) $n + 42$
 - n
- b) $n + 1000000$
 - n
- c) $n^2 + n + 42$
 - n^2
- d) $n^2 + 1000000n + 5$
 - n^2
- e) $n^2 + n^3$
 - n^3
- f) $n + \log n + n \log n$
 - $n \log n$
- g) $n + (\log n)^5 + n \log n$
 - $n \log n$

LEC27 - Big Oh Notations Expanded

- h) $2^n + n^2$
 - 2^n
- i) $2^n + n^{200}$
 - 2^n

Time Complexity of Algorithms

- Measure of how efficient an algorithm is, given input size of n
- We measure time complexity in the order of number of operations an algorithm uses
 - **Big-O:** An upper bound on the number of operations an algorithm conducts to solve a problem with input size of n
 - **Big-Theta:** A tighter bound on the number of operations an algorithm conducts to solve a problem of input size n
- We will be looking at the worst case analysis (number of operations in the worst case)
 - Don't confuse Big-O and Big-Theta with best/average/worst case analysis

Example

```
def max(list):  
    max = list[0]  
    for i in range(len(list)):  
        if max < list[i]:  
            max = list[i]  
    return max  
  
# What is the time complexity of this algorithm?
```

- n comparisons are made by the for loop
- Therefore, the time complexity of this algorithm is $\theta(n)$

Example

```
def max2(list):
    max = list[0]
    i = 1
    while i < len(list):
        if max < list[i]:
            max = list[i]
        i = i + 1
    return max
```

What is the time complexity of this algorithm?

- n comparisons are made by the while loop
 - Therefore, the time complexity of this algorithm is $\theta(n)$
-

Example

```
def blah(n):
    n = 148 * n + n**(165)
    print ("n is equal to {}". format(n))
    if n > 1000:
        print ("n is over 1000")
    elif n > 100:
        print ("n is over 100")
    else:
        print ("n is under 100")
```

What is the time complexity of this algorithm?

- If statements are constant time, along with print statements
- Therefore, the time complexity of this algorithm is $\theta(1)$

Example

```
def blah3(n):
    n = 148 * n + n**(165)
    print ("n is equal to {}". format(n))
    if n > 1000:
        print ("n is over 1000")
    elif n > 100:
        for i in range(n):
            print ("n is over 100")
    else:
        print ("n is under 100")

# What is the time complexity of this algorithm?
```

- In the vast majority of cases, the first if statement will execute, which is constant time
 - We need to consider the cases when n gets very large, which will certainly be over 1000 after exponentiation
 - Therefore, the time complexity of this algorithm is $\theta(1)$
-

Example

```
sum = 0
for i in range(n//2):
    sum += i * n
for i in range(n//2):
    for j in range(n**2):
        sum += i * j

# What is the time complexity of this algorithm?
```

- Code runs $\frac{1}{2}n + \frac{1}{2}n \cdot n^2$ times
- Therefore, the time complexity of this algorithm is $\theta(n^3)$

Example

```
sum = 0
if n % 2 == 0:
    for i in range(n*n):
        sum += 1
else:
    for i in range(n+100):
        sum += i

# What is the time complexity of this algorithm?
```

- If n is even, then the code runs n^2 times
 - If n is odd, then the code runs n times
 - Since we are considering the worst-case time complexity, we consider n^2
 - Therefore, the time complexity of this algorithm is $\theta(n^2)$
-

Example

```
i, sum = 0, 0
while i ** 2 < n:
    j = 0
    while j ** 2 < n:
        sum += i * j
        j += 1
    i += 1

# What is the time complexity of this algorithm?
```

- Outer loop runs $\sqrt{n}$ times
 - $i^2 < n \Rightarrow i < \sqrt{n}$
- Inner loop runs $\sqrt{n}$ time as well
- Therefore, the algorithm runs $\sqrt{n} \cdot \sqrt{n} = n$ times
- Therefore, the time complexity of this algorithm is $\theta(n)$

Example

```
count = 0
while n > 1:
    n = n // 2
    count = count + 1
print(count)

# What is the time complexity of this algorithm?
```

- Loop executes $\log_2 n = \log n$ times, since n is halved each iteration
 - Therefore, the time complexity of this algorithm is $\theta(\log n)$
-

Example

```
count, i = 0, 1
while i < n:
    i = i * 2
    count = count + 1
print(count)

# What is the time complexity of this algorithm?
```

- Loop executes $\log_2 n = \log n$ times, since i being doubled each iteration is the same as n being halved each iteration
- Therefore, the time complexity of this algorithm is $\theta(\log n)$